



**Universidad
Tecnológica
del Perú**

“Aplicación de Desing Thinking para la mejora de los trámites municipales”

Integrantes:

Beraun García, Leonardo Ángel – 100%

Macedo Salas, Edhu Anthony – 100%

Matos Chuquino, Jesús Presencio – 100%

Asistente Virtual:

Manuelito

Curso:

Diseño de Productos y Servicios

Docente:

Guevara Jiménez, Jorge Alfredo

Sección:

24896

2026 – I

INDICE

1. Prototipado rápido y Pruebas de Concepto	1
1.1. Historia de Usuario Validada (HU)	1
1.2. Criterios Given–When–Then (Criterios de Aceptación).....	1
1.3. Prototipo wireframe tipo Balsamiq (Estructura UI/UX)	2
1.4. Diagrama de Arquitectura	3
1.5. Estructura del Repositorio GitHub	4
1.6. Selección Tecnológica Justificada.....	5
1.7. Código Funcional Inicial	5
1.8. Enlaces de Entrega	6
2. Iteración y mejora continua.....	7
2.1. Explicación conceptual del Ciclo Deming (PDCA)	7
2.2. Objetivo de la práctica	7
2.3. Caso de estudio	8
2.4. Requisitos técnicos de la aplicación	9
2.5. Estructura GitHub esperado.....	10
2.6. Historia de Usuario Principal.....	11
2.7. Criterios de aceptación.....	12
2.8. Wireframe simplificado	13
2.9. Organización de actividades (90 minutos)	14
2.10. Cuadro Integrado de Mejora Continua PDCA	15
2.11. Lista de verificación de revisión GitHub.....	17
2.12. Evidencias requeridas	17
2.13. Reflexión final.....	19
3. Diseño de Experiencia de Usuario (UX). Diseño de Interfaces y Visualización de Conceptos	20
3.1. Diseño del mockup con 10 principios de usabilidad de Nielsen	22
3.2. Diseño del mockup aplicado con la psicología de colores	26
3.3. Diseño de la carpeta github con el MVP1 completo	27

3.4. Enlace de video funcional del MVP1 con 7 registros continuos.....	28
3.5. Reporte de validación con IA sobre: carpeta github y la arquitectura hexagonal y base de datos en capas con integración de la IA	28
3.6. Reporte de validación con IA sobre: carpeta github cumple con los 10 principios de usabilidad y la paleta de colores.....	29
3.7. Reporte de validación con IA sobre: Lista de cotejo de mejora continua PDCA para video, wireframe, historia de usuario, arquitectuta y github	29

1. Prototipado rápido y Pruebas de Concepto (iteración 9)

1.1. Historia de Usuario Validada (HU)

Se ha seleccionado la HU de mayor valor para el ciudadano, la cual integra el requerimiento de la Inteligencia Artificial desde la fase inicial (MVP):

- HU-02
- Título: Visualización detallada del trámite con Asistente de Comprensión IA.
- Descripción: Como ciudadano peruano, quiero visualizar de forma clara y 100% pública el detalle de un trámite municipal (costos, tiempos, requisitos), incluyendo un resumen en lenguaje sencillo (generado por IA) y un chatbot interactivo, para comprender el proceso burocrático sin necesidad de contratar a un tramitador.
- Justificación de Negocio: Reduce la fricción cognitiva y la brecha digital.

1.2. Criterios Given–When–Then (Criterios de Aceptación)

Escenario 1: Interacción con el Asistente Virtual (Chatbot IA)

Given Dado que el ciudadano se encuentra en la pantalla de detalle del trámite "Licencia de Funcionamiento" y tiene dudas sobre un requisito técnico. When Cuando el usuario escribe la pregunta "*¿El croquis se puede hacer a mano?*" en el panel lateral del Chatbot y presiona enviar. Then Entonces el sistema procesa la consulta y el Asistente IA responde de manera contextual (ej. "*Sí, puedes hacerlo a mano alzada marcando las calles principales*"), simulando un tiempo de procesamiento humano.

Escenario 2: Visualización de Datos Oficiales

Given: Dado que el usuario seleccionó un trámite desde el Catálogo Principal.
When: Cuando carga la vista de detalle. Then: Entonces la pantalla debe dividirse mostrando un 70% para la información oficial y un 30% fijo para el Asistente IA.

1.3. Prototipo wireframe tipo Balsamiq (Estructura UI/UX)

El diseño respeta el principio de "Carga Cognitiva Reducida", dividiendo la interfaz en dos bloques lógicos:

The wireframe illustrates a web interface for a commercial license application, designed to reduce cognitive load by separating the main application details from an AI assistant. The interface is divided into two main vertical sections.

Left Section (Main Application Details):

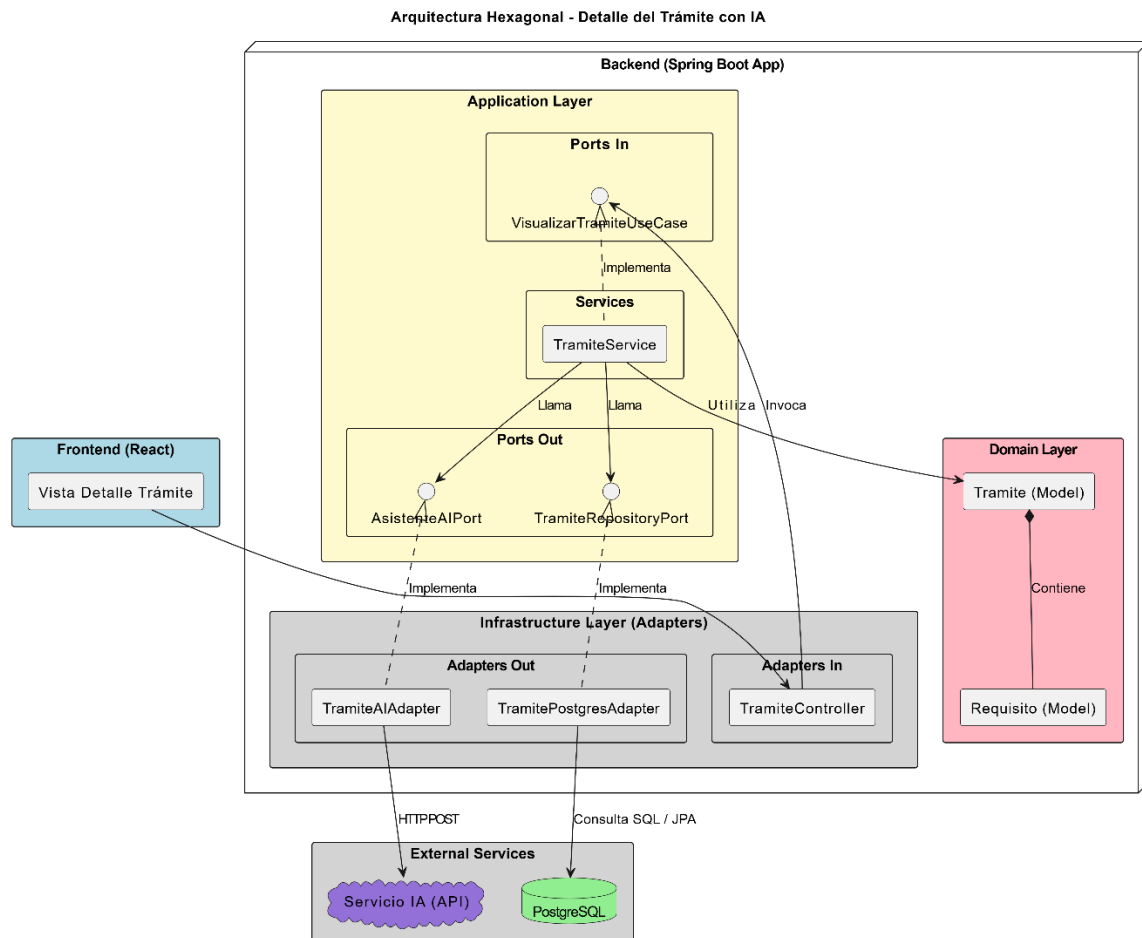
- Header:** Includes a menu icon and a link "Volver al Catálogo".
- Title:** "Trámite: Licencia de Funcionamiento Comercial".
- Metadata:** "Costo: S/ 150", "días hábiles: 5 días", "tipo: Virtual".
- AI Summary:** A highlighted box containing the text: "★ Resumen Inteligente IA: Este trámite te permite abrir locales menores a 100m2. Solo necesitas tu DNI, contrato de alquiler y un extintor vigente."
- Official Requirements:** A section titled "Requisitos Oficiales:" with three checkboxes:
 - ☐ DNI vigente
 - ☐ Declaración Jurada
 - ☐ Croquis de Distribución

Right Section (AI Assistant):

- Header:** "★ Asistente de IA del Trámite".
- AI Message:** A highlighted box containing the text: "IA: El croquis de distribución puede ser dibujado a mano alzada, no necesitas un arquitecto para locales pequeños."
- Input Field:** A text input field with the placeholder text: "Pregúntale a la IA sobre este trámite... Ej: ¿Cómo hago el croquis?".
- Submit Button:** A black button labeled "Enviar".

1.4. Diagrama de Arquitectura

El backend está estrictamente basado en Arquitectura Hexagonal (Puertos y Adaptadores) para asegurar la independencia de la tecnología y el uso temporal de la IA simulada.



1.5. Estructura del Repositorio GitHub

La organización del proyecto separa claramente las capas y las responsabilidades:

muniGo/

```
├─ database/
|   └─ init.sql # Script SQL (Tablas y Tramites Semilla)
├─ backend/ # Spring Boot (Java)
|   └─ src/main/java/com/muni/tramites/
|       ├─ domain/ # Entidades core puras
|       ├─ application/ # Interfaces (Ports) y Casos de Uso
|       └─ infrastructure/ # Controllers, Repositories (JPA) y Adaptador IA
|
└─ frontend/ # React + Vite
    ├─ src/
    |   ├─ components/
    |   |   ├─ CatalogoTramites.jsx # Vista H1
    |   |   └─ DetalleTramite.jsx # Vista H2
    |   └─ PanelChatbot.jsx # Componente IA Simulado
    └─ App.jsx # Enrutador principal
    └─ tailwind.config.js
    └─ package.json
```

1.6. Selección Tecnológica Justificada

- **Frontend (React + Tailwind CSS):** Se eligió React por su modelo de componentes, lo que permite crear el Panel de Chatbot de forma modular y mantener su estado (historial de mensajes) sin recargar la página. Tailwind CSS permite diseñar interfaces modernas rápidamente, vital para el "Design Thinking".
- **Backend (Java + Spring Boot):** Es el estándar empresarial por excelencia. Facilita la implementación rigurosa de la Arquitectura Hexagonal y la Inversión de Dependencias (IoC).
- **Base de Datos (PostgreSQL):** Un motor relacional robusto. Se configuró localmente (spring.jpa.show-sql=true) para auditar la inserción y lectura de requisitos legales garantizando integridad.
- **IA Simulado (Mock Adapter en Backend/Frontend):** Para efectos de este MVP, la IA se inyecta mediante *Mocks* usando demoras asíncronas (setTimeout) y detección de palabras clave, ahorrando costos de API pero validando la Arquitectura.

1.7. Código Funcional Inicial

A continuación, se muestra el fragmento clave del componente React que gestiona el chatbot inteligente simulado:

```
○ ○ ○  
  
useEffect(() => {  
  if (tramite) {  
    setMessages([  
      {  
        id: 1,  
        sender: 'ia',  
        text: `¡Hola! Soy tu Asistente Municipal IA.  
        Estoy aquí para ayudarte específicamente con tu consulta sobre "${tramite.nombre}".  
        ¿Qué duda tienes sobre los requisitos o el proceso?`  
      }  
    ]);  
  }  
}, [tramite]);  
  
useEffect(() => {  
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });  
}, [messages, isTyping]);
```



```

const handleSubmit = (e) => {
  e.preventDefault();
  if (!input.trim()) return;

  const userMessageText = input;
  const userMessage = {
    id: Date.now(),
    sender: 'user',
    text: userMessageText
  };

  setMessages((prev) => [...prev, userMessage]);
  setInput('');
  setIsTyping(true);

  setTimeout(() => {
    let replyText = '';
    const normalizedQuery = userMessageText.toLowerCase();

    if (normalizedQuery.includes('croquis') || normalizedQuery.includes('dibujo')) {
      replyText = 'El croquis de distribución puede ser dibujado a mano alzada, no necesitas un arquitecto para locales pequeños. Solo marca calles principales e ingresos.';
    } else if (normalizedQuery.includes('fut') || normalizedQuery.includes('formulario')) {
      replyText = 'El Formulario Único de Trámite (FUT) es totalmente gratuito y se pide en mesa de partes para iniciar la solicitud de "${tramite.nombre}.";';
    } else if (normalizedQuery.includes('tiempo')) {
      replyText = 'Este trámite (${tramite.nombre}) demora aproximadamente ${tramite.tiempoEstimado} hábiles una vez entregados todos los requisitos oficiales.';
    } else {
      replyText = 'Para resolver dudas adicionales sobre el trámite de "${tramite.nombre}", te invitamos a acercarte a la ventanilla de atención al ciudadano en el palacio municipal.';
    }
  });
}

```

1.8. Enlaces de Entrega

- Enlace al Repositorio GitHub: <https://github.com/Estudiante-leonardo/Muni-Go.git>
- Video Demostrativo del Funcionamiento: [Video-Productos.mp4 - Google Drive](#)

2. Iteración y mejora continua (iteración 10)

2.1. Explicación conceptual del Ciclo Deming (PDCA)

El ciclo Deming o ciclo Plan-Do-Check-Act (PDCA) es una metodología de mejora continua aplicada para optimizar procesos, productos y servicios. En el contexto de nuestro proyecto Muni-Go, lo aplicamos de la siguiente manera:

- **PLAN (Planificar):**

Identificar la fricción que sufren los ciudadanos al buscar trámites (ej. no saber los pasos a seguir o dónde descargar formularios) y planificar soluciones de UI/UX y arquitectura backend para resolverlo.

- **DO (Hacer):**

Desarrollar e integrar las mejoras en el código. Esto incluye la creación de nuevos componentes en React (cajas de descarga, líneas de tiempo, selectores de distrito) y la adaptación de los controladores y servicios en Spring Boot.

- **CHECK (Verificar):**

Validar si las nuevas funcionalidades en la interfaz realmente reducen la carga cognitiva del usuario y comprobar si el Asistente IA (simulado) responde adecuadamente sin romper el flujo de la aplicación.

- **ACT (Actuar):**

Estandarizar los componentes de UI exitosos para reutilizarlos en otros trámites (ej. un componente universal de "Pasos") y consolidar la arquitectura hexagonal del backend para que la futura integración con una API de IA real sea limpia y sin fricciones.

2.2. Objetivo de la práctica

Aplicar de forma práctica el ciclo PDCA para estructurar y documentar la mejora continua de la plataforma "Muni-Go" (Multi-distrito).

El objetivo específico de esta iteración trasciende la simple codificación: buscamos perfeccionar la Experiencia de Usuario (UX) de la vista de

catálogo y detalle de trámites, evolucionando de un prototipo informativo básico a una interfaz altamente interactiva y accionable (que incluya geolocalización por distrito, descarga de formatos FUT y flujos secuenciales). Asimismo, se busca validar el comportamiento y la utilidad de un Asistente de IA (actualmente en fase de simulación) para asegurar que la arquitectura tecnológica (React + Spring Boot) esté lista, testeada y modularizada para la inyección del modelo de Inteligencia Artificial real en los próximos sprints.

2.3. Caso de estudio

La aplicación desarrollada (Muni-Go) permite:

- Seleccionar y cambiar la municipalidad de interés en tiempo real.
- Visualizar información detallada de trámites (costos, tiempos, requisitos).
- Descargar formatos oficiales (FUT) y visualizar los pasos a seguir de forma secuencial.
- Ubicar la sede física correspondiente al trámite seleccionado.
- Interactuar con un Asistente IA (simulado en esta iteración mediante retardos asíncronos) para resolver dudas de manera conversacional y en lenguaje sencillo.

2.4. Requisitos técnicos de la aplicación

La solución implementa una Arquitectura Hexagonal en el backend (Spring Boot) y un frontend modular (React), dividiéndose en:

Capa	Responsabilidad
Vista (Frontend)	Interfaces de usuario en React (Catálogo, Detalle, Panel Chatbot).
Controlador / Adapters In	Recepción de solicitudes REST en Spring Boot.
Modelo / Domain	Lógica de negocio pura (Entidades: Trámite, Requisito, Municipalidad).
Base de Datos / Adapters Out	Persistencia de datos mediante PostgreSQL y JPA.
Integración IA (Simulada)	Servicios en el frontend/backend que emulan respuestas de IA mediante retardos asíncronos y análisis de palabras clave.

2.5. Estructura GitHub esperado

Para garantizar el cumplimiento de la arquitectura hexagonal y el modularidad del frontend, la organización del repositorio Muni-Go es la siguiente:

Muni-Go/

```
├─ backend/ # Servidor y API REST (Spring Boot)
|   └─ src/main/java/com/muni/tramites/
|       └─ domain/ # Lógica de negocio pura
|           └─ model/ # Entidades del negocio
|               └─ ports/ # Interfaces para los adaptadores
|                   └─ application/ # Casos de uso y servicios orquestadores
|                       └─ infrastructure/ # Implementación de adaptadores
|                           └─ adapters/ # JPA Repositories, REST Controllers
|                               └─ config/ # Configuración de BD y Beans
|                                   └─ Application.java # Clase principal
|                                       └─ application.properties # Configuración de la aplicación
├─ frontend/ # Interfaz de Usuario (React + Vite)
|   └─ src/
|       └─ assets/ # Imágenes y recursos estáticos
|           └─ components/ # Componentes reutilizables
|               └─ layout/ # Navbars y Footers
|                   └─ tramite/ # Tarjetas y detalles de trámites
|                       └─ chatbot/ # PanelChatbot y Loader
```

```
| | └─ contexts/ # Estados globales
| | └─ pages/ # Vistas principales
| | └─ routes/ # Enrutamiento de la aplicación
| | └─ services/ # api.js e iaService.js
| | └─ utils/ # Funciones auxiliares
| | └─ App.jsx # Componente raíz
| └─ package.json
└─ database/ # Configuración de base de datos
  └─ scripts/
    └─ init.sql # Tablas y datos semilla
└─ docs/ # Documentación del proyecto
  └─ Informe_APF.pdf
└─ README.md # Guía técnica y despliegue
```

2.6. Historia de Usuario Principal

Historia de Usuario (HU-03: Visualización Integral y Asistencia)

- **Como** ciudadano peruano.
- **Necesito** consultar los requisitos, formatos descargables, pasos a seguir y ubicación física de un trámite, además de poder seleccionar mi municipalidad específica.
- **Para** prepararme correctamente antes de ir presencialmente y resolver mis dudas inmediatas mediante un asistente de IA (simulado).

2.7. Criterios de aceptación

- **Escenario 1 - Cambio de Municipalidad dinámico:**
 - *Given* que el usuario se encuentra en la vista de detalle de un trámite.
 - *When* selecciona una nueva municipalidad en el botón superior derecho.
 - *Then* el sistema actualiza la vista mostrando la dirección ("¿Dónde ir?") y los costos/requisitos específicos de ese distrito.
- **Escenario 2 - Descarga de Formatos y Pasos:**
 - *Given* que el ciudadano revisa los requisitos de la Licencia de Funcionamiento.
 - *When* se desplaza hacia abajo en la pantalla principal.
 - *Then* el sistema muestra una sección clara con el "Formato Único de Trámite" habilitado para descarga y una línea de tiempo con los pasos 1, 2 y 3 a seguir.
- **Escenario 3 - Interacción con IA Simulada:**
 - *Given* que el paciente tiene dudas sobre cómo hacer un croquis.
 - *When* escribe la pregunta en el panel lateral derecho del Asistente y presiona enviar.
 - *Then* el sistema muestra un indicador de "escribiendo..." y luego de unos segundos, devuelve una respuesta contextual simulada resolviendo la duda.

2.8. Wireframe simplificado

Área	Componentes
Encabezado	Botón de retorno, Nombre del sistema, Botón Selector de Municipalidad.
Cabecera del Trámite	Título, Costo, Días Hábiles, Tipo de trámite, Resumen Inteligente IA.
Cuerpo Principal	Checkbox de Requisitos Oficiales.
Nuevas Secciones (Mejora)	Caja de "Formatos para descargar" (FUT), Línea de tiempo "Pasos a seguir", Caja de ubicación "¿Dónde ir?".
Panel Lateral (Derecho)	Asistente de IA del Trámite (Chatbot) con historial de mensajes e input de texto.

A Web Page

← → ↻

https://MuniGo/Descripcion_Catalogo

☰

← Volver al Catálogo

MuniVirtual

Trámite: Licencia de Funcionamiento Comercial

Costo: S/ 150 días hábiles: 5 días tipo: Virtual

★

Resumen Inteligente IA: Este trámite te permite abrir locales menores a 100m2. Solo necesitas tu DNI, contrato de alquiler y un extintor vigente.

Requisitos Oficiales:

☐ DNI vigente
☐ Declaración Jurada
☐ Croquis de Distribución

📄

Formatos para descargar

Imprime y llena estos documentos desde casa para evitar colas o buscar copias el mismo día.

Formato Único de Trámite (FUT)

Documento PDF · 120 KB

👁️ ⬇️

Pasos a seguir:

1

Preparar Expediente
 Junta todos los requisitos y llena los formatos descargados en un folder.

2

Acercarse a Sede
 Dirígete a la Municipalidad de Carabayllo. Si tu trámite tiene costo (S/ 120), pasa primero por caja.

3

Mesa de Partes
 Entrega tus documentos. Te asignarán un número para que consultes luego el resultado.

📍 ¿Dónde ir?

Municipalidad de Carabayllo

Plaza Central del distrito

Horario: L-V de 8:00 AM a 4:30 PM

★ Asistente de IA del Trámite

IA: El croquis de distribución puede ser dibujado a mano alzada, no necesitas un arquitecto para locales pequeños.

Pregúntale a la IA sobre este trámite... Ej: ¿Cómo hago el croquis?

Enviar

2.9. Organización de actividades (90 minutos)

Actividad	Tiempo
Explicación conceptual PDCA	10 min
Revisión de aplicación y GitHub	15 min

14

Desarrollo de cuadro PDCA	40 min
Presentación de mejoras	15 min
Retroalimentación	10 min

2.10. Cuadro Integrado de Mejora Continua PDCA

PLAN (Problema detectado)	DO (Mejora aplicada)	CHECK (Verificación)	ACT (Acción permanente)
Falta de especificidad local: Los trámites varían por distrito, pero la UI mostraba datos genéricos.	Botón de Municipalidad: Se añadió un selector de municipalidad en la esquina superior derecha del UI.	Al cambiar de distrito, la información de ubicación y requisitos se actualiza.	Implementar un estado global (Context API en React) para manejar la municipalidad activa en toda la app.
Desconocimiento de anexos: Los usuarios no sabían de dónde sacar los formularios (ej. FUT).	Sección Descargas: Se integró el bloque "Formatos para descargar" en el wireframe y UI.	Los usuarios pueden identificar visualmente el ícono de descarga del documento PDF.	Estandarizar una tabla en PostgreSQL para vincular URLs de documentos a cada trámite.
Incertidumbre en el flujo: Tener los requisitos no explica el orden de las acciones a realizar.	Timeline de Pasos: Se diseñó la sección "Pasos a seguir" (1. Preparar, 2. Acercarse, 3. Mesa de partes).	La lectura secuencial reduce la confusión del ciudadano.	Mantener un componente React reutilizable <Stepper /> para los pasos de cualquier trámite.
Problema de ubicación: El usuario preparaba sus papeles pero no sabía a qué edificio ir.	Geolocalización: Se añadió la caja "¿Dónde ir?" vinculada al distrito seleccionado.	Se muestra claramente la dirección física (Ej. Plaza Central de Carabayllo).	Agregar futuras integraciones con Google Maps o Leaflet en este componente.
Lógica de IA mezclada en UI: El componente del Chatbot contenía demasiada lógica de simulación (setTimeout).	Servicio Frontend: Se movió la lógica simulada a un archivo independiente iaService.js.	El componente PanelChatbot.jsx queda limpio y solo maneja renderizado.	Mantener esta separación de responsabilidades; facilitará cambiar el mock por una API real luego.
Experiencia visual del Chatbot: La IA respondía instantáneamente, sintiéndose poco natural.	Indicador de "Escribiendo": Se añadió un estado de carga UI	La experiencia interactiva es mucho más realista y amigable.	Estandarizar un componente <LoaderChat /> para todas las interacciones futuras con IA.

	simulando el tiempo de procesamiento.		
Envío de mensajes vacíos: El usuario podía hacer clic en "Enviar" sin escribir nada en el chat.	Validación de Input: Se implementó una restricción en el formulario del chatbot en React.	Ya no se procesan ni renderizan burbujas de texto en blanco.	Aplicar validaciones de campos (formularios) en toda la aplicación de manera estándar.
Arquitectura de Software (Backend): Riesgo de acoplamiento al conectar la BD.	Adaptadores Hexagonales: Se crearon puertos (TramiteRepositoryPort) para aislar la lógica.	El controlador REST no toca la BD directamente, usa el servicio de dominio.	Respetar estrictamente los principios de Arquitectura Hexagonal en los próximos sprints.
Historia de Usuario (Formato): Las historias previas no seguían el estándar BDD exigido en la guía.	Refinamiento BDD: Se reescribió la HU usando la plantilla Rol-Necesito-Para y Given-When-Then.	La historia ahora cubre las nuevas secciones (descargas, municipalidad, pasos).	Utilizar siempre este estándar para definir los requisitos en las siguientes iteraciones.
Organización de Repositorio: Mezcla de componentes en una sola carpeta en frontend.	Reestructuración de carpetas: Se organizó /components separando /layout, /chatbot y /tramite.	El código es mucho más fácil de navegar y mantener por el equipo.	Documentar esta estructura en el README.md del repositorio oficial.

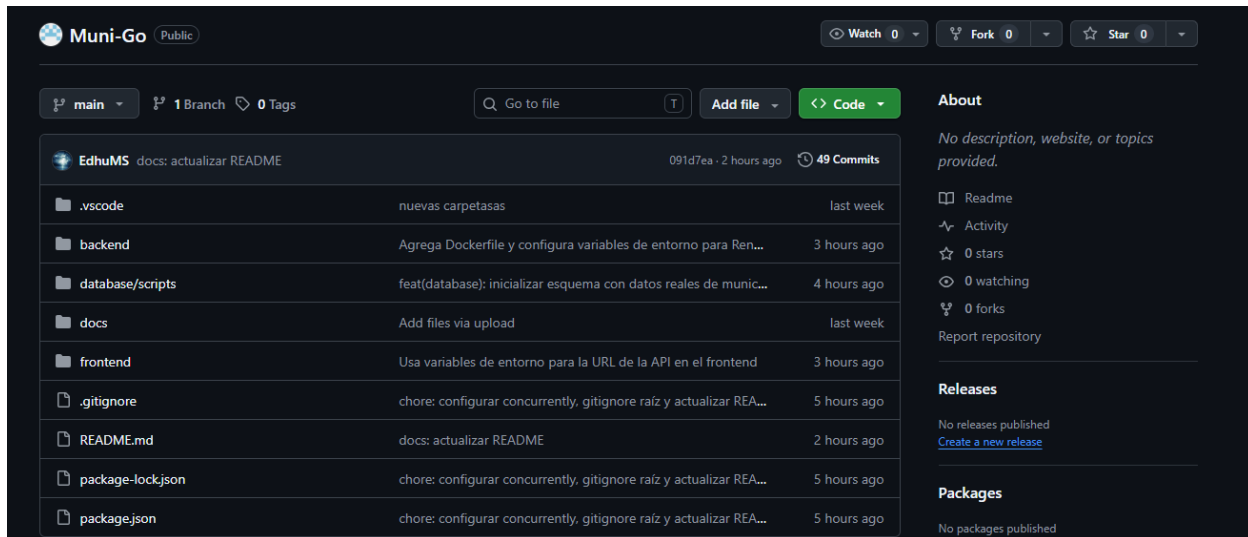
2.11. Lista de verificación de revisión GitHub

Elemento a revisar	Cumple
Carpeta controllers (Adapters In)	Sí
Carpeta models (Domain)	Sí
Carpeta views (Components React)	Sí
Carpeta database (Infra/PostgreSQL)	Sí
Carpeta integration (iaService / Ports)	Sí
Separación MVC/Hexagonal correcta	Sí
Clase conexión BD	Sí
Clase integración IA (Servicio Simulado)	Sí
README técnico	Sí
Commits organizados	Sí

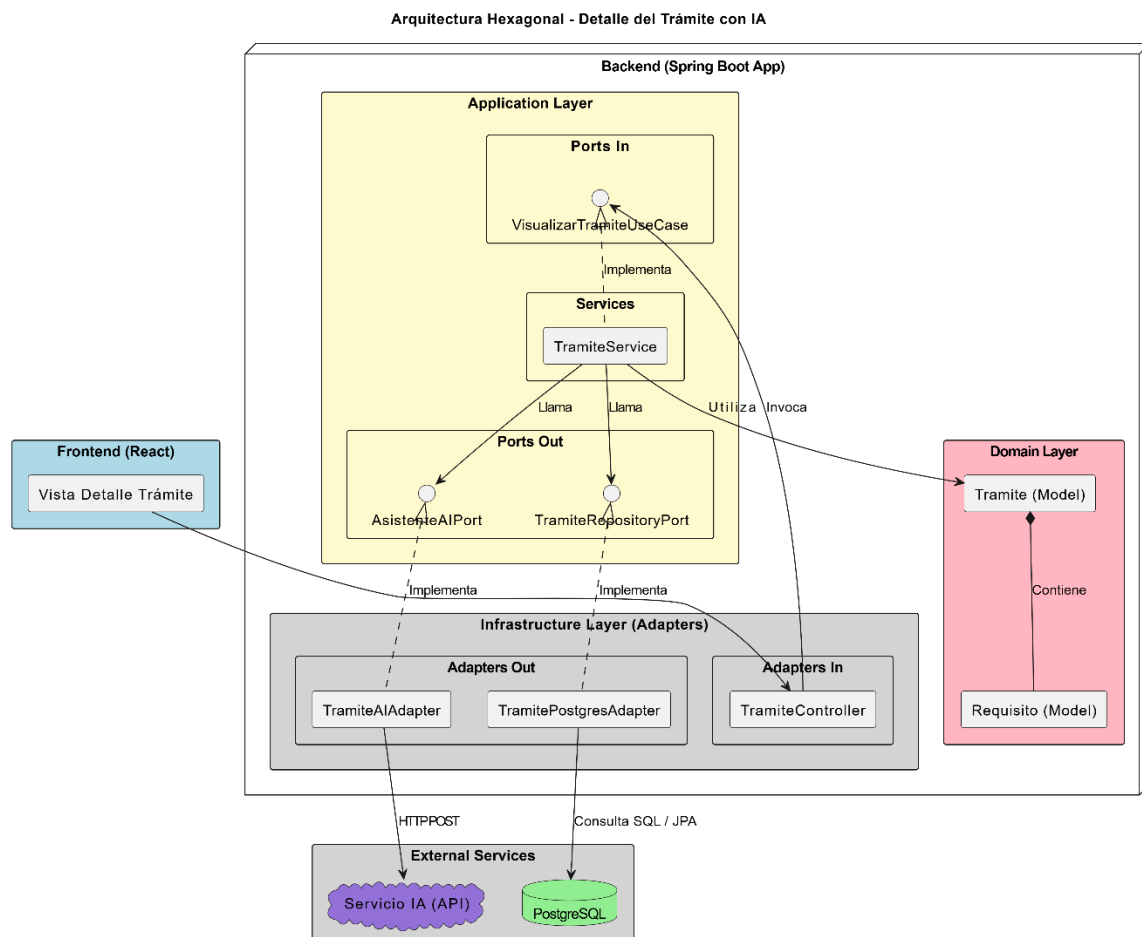
2.12. Evidencias requeridas

- **URL del repositorio:** <https://github.com/Estudiante-leonardo/Muni-Go.git>
- **URL de la web funcional:** <https://muni-go-a6xj.onrender.com/>

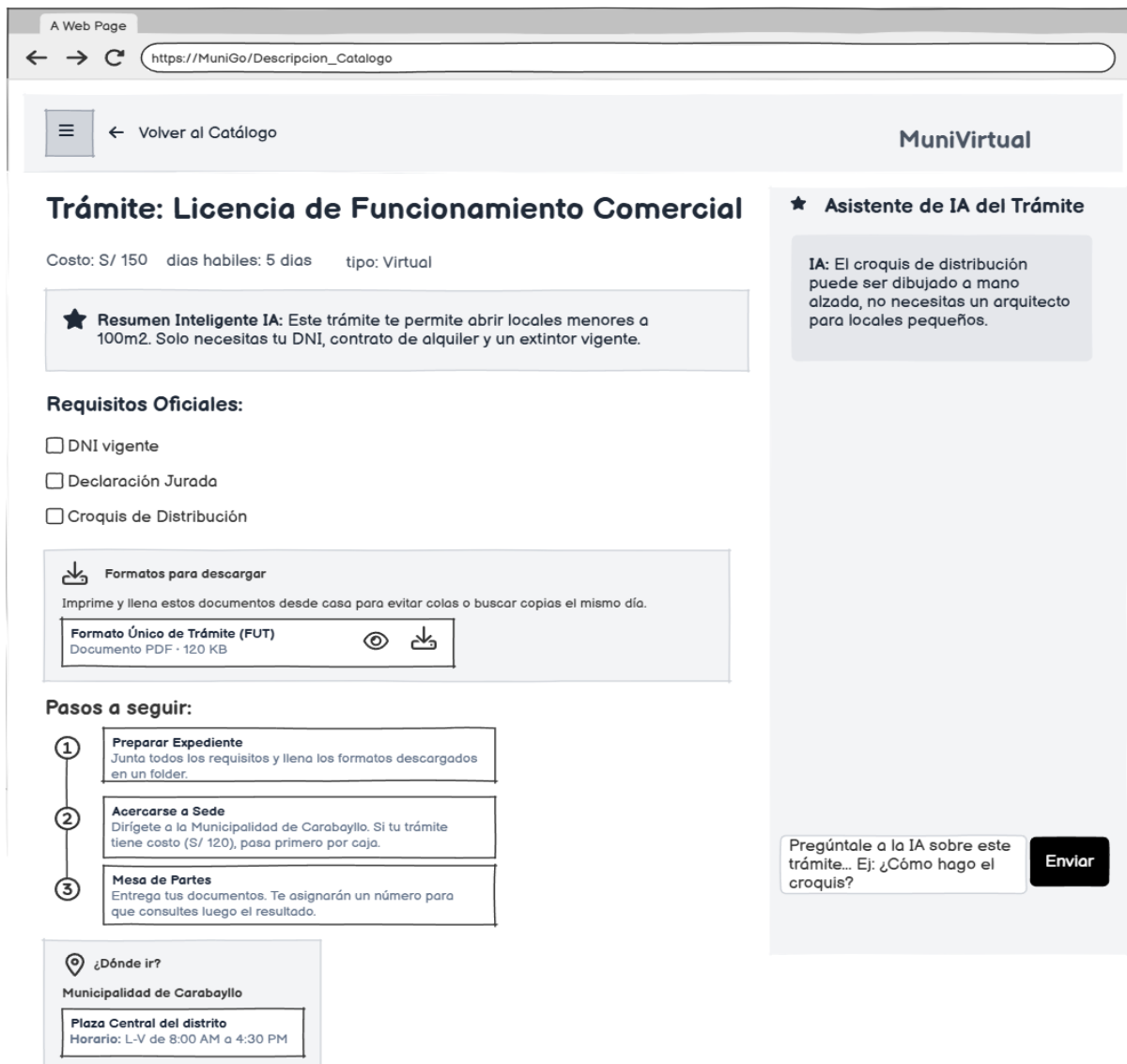
- Capturas de GitHub:



- Capturas de arquitectura:



- Capturas del wireframe:



2.13. Reflexión final

La aplicación del ciclo PDCA nos ha permitido darnos cuenta de que la mejora continua no solo se aplica al código backend, sino fuertemente a la Experiencia del Usuario (UX). Al evaluar nuestro primer wireframe, detectamos que brindar solo requisitos no era suficiente. Planificamos e implementamos mejoras tangibles: descarga de formatos, flujos de pasos y geolocalización vinculada a un selector de municipalidades. Aislar la IA en un servicio simulado nos permite tener un MVP funcional y testeable hoy, preparando el terreno arquitectónico para conectar una API real de Inteligencia Artificial en el siguiente ciclo, asegurando siempre un software escalable y centrado en el ciudadano.

3. Diseño de Experiencia de Usuario (UX). Diseño de Interfaces y Visualización de Conceptos (iteración 11)

3.1. Actualización del wireframe

MuniGo

[←](#) [→](#) [↻](#) <https://MuniGo/Tramites/#>

☰

[← Volver al Catálogo](#)

carabayillo ▾

Trámite: Licencia de Funcionamiento Comercial

Costo: S/ 150 días hábiles: 5 días categorías: Licencias

★ **Resumen Inteligente IA:** Este trámite te permite abrir locales menores a 100m². Solo necesitas tu DNI, contrato de alquiler y un extintor vigente.

Requisitos Oficiales:

- ☐ DNI vigente
- ☐ Declaración Jurada
- ☐ Croquis de Distribución

 **Formatos para descargar**

Imprime y llena estos documentos desde casa para evitar colas o buscar copias el mismo día.

Formato Único de Trámite (FUT)
Documento PDF · 120 KB

Pasos a seguir:

- Preparar Expediente**
Junta todos los requisitos y llena los formatos descargados en un folder.
- Acercarse a Sede**
Dirígete a la Municipalidad de Carabayillo. Si tu trámite tiene costo (S/ 120), pasa primero por caja.
- Mesa de Partes**
Entrega tus documentos. Te asignarán un número para que consultes luego el resultado.

 **¿Dónde ir?**

Municipalidad de Carabayillo

Plaza Central del distrito
Horario: L-V de 8:00 AM a 4:30 PM

 **Asistente de IA del Trámite**

IA: El croquis de distribución puede ser dibujado a mano alzada, no necesitas un arquitecto para locales pequeños.

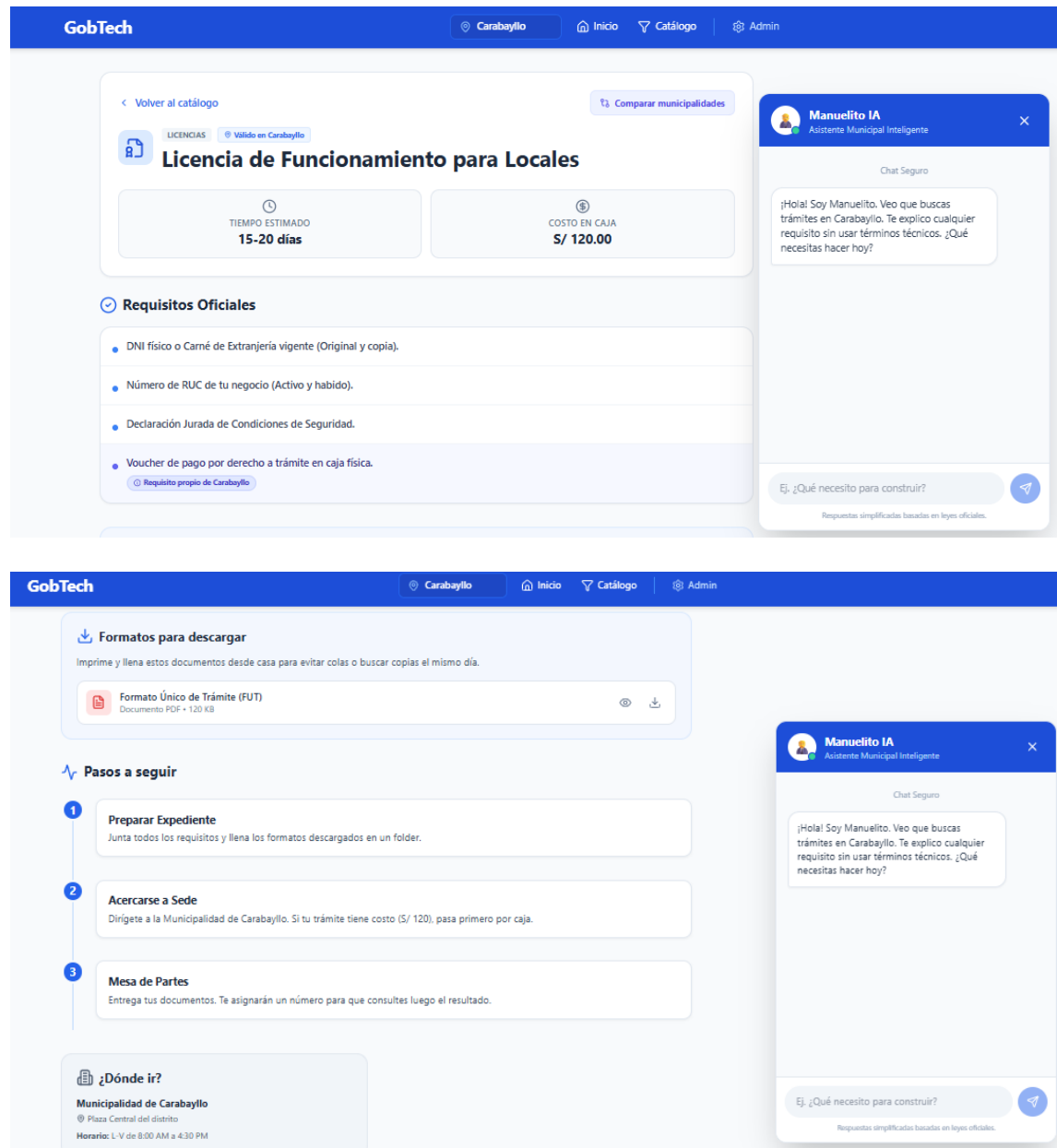
Pregúntale a la IA sobre este trámite... Ej: ¿Cómo hago el croquis?

3.2. Actualización del PDCA

PLAN	DO	CHECK	ACT	responsabilidad	fecha de selección
Falta de especificidad local: Los trámites varían por distrito, pero la UI mostraba datos genéricos.	Botón de Municipalidad: Se añadió un selector de municipalidad en la esquina superior derecha del UI.	Al cambiar de distrito, la información de ubicación y requisitos se actualiza.	Implementar un estado global (Context API en React) para manejar la municipalidad activa en toda la app.	Edhu	29/05/2026
Desconocimiento de anexos: Los usuarios no sabían de dónde sacar los formularios (ej. FUT).	Sección Descargas: Se integró el bloque "Formatos para descargar" en el wireframe y UI.	Los usuarios pueden identificar visualmente el ícono de descarga del documento PDF.	Estandarizar una tabla en PostgreSQL para vincular URLs de documentos a cada trámite.	Edhu	29/05/2026
Incertidumbre en el flujo: Tener los requisitos no explica el orden de las acciones a realizar.	Timeline de Pasos: Se diseñó la sección "Pasos a seguir" (1. Preparar, 2. Acercarse, 3. Mesa de partes).	La lectura secuencial reduce la confusión del ciudadano.	Mantener un componente React reusable <Stepper /> para los pasos de cualquier trámite.	Edhu	29/05/2026
Falta de información rápida: El usuario tenía dudas comunes, pero la app no tenía sección de FAQ (solo un enlace inactivo), forzándolo a abandonar la página o acudir presencialmente.	Componente FAQ en Modal: Se implementó FaqModal.jsx (acordeón de preguntas) y se activó mediante botones estratégicos en el Footer y el Sidebar.	Al hacer clic en "Preguntas Frecuentes", el ciudadano accede instantáneamente a las respuestas sin salir de la vista actual ni recargar la página.	Estandarizar el uso de Modales superpuestos para información complementaria (FAQ, Términos).	Leonardo	3/06/2026
Problema de ubicación: El usuario preparaba sus papeles pero no sabía a qué edificio ir.	Geolocalización: Se añadió la caja "¿Dónde ir?" vinculada al distrito seleccionado.	Se muestra claramente la dirección física (Ej. Plaza Central de Carabaylo).	Agregar futuras integraciones con Google Maps o Leaflet en este componente.	Jesus	31/05/2026
Lógica de IA mezclada en UI: El componente del Chatbot contenía demasiada lógica de simulación (setTimeout).	Servicio Frontend: Se movió la lógica simulada a un archivo independiente iaService.js.	El componente PanelChatbot.jsx queda limpio y solo maneja renderizado.	Mantener esta separación de responsabilidades; facilitará cambiar el mock por una API real luego.	Leonardo	31/05/2026
Experiencia visual del Chatbot: La IA respondía instantáneamente, sintiéndose poco natural.	Indicador de "Escribiendo": Se añadió un estado de carga UI simulando el tiempo de procesamiento.	La experiencia interactiva es mucho más realista y amigable.	Estandarizar un componente <LoaderChat /> para todas las interacciones futuras con IA.	Leonardo	31/05/2026
Envío de mensajes vacíos: El usuario podía hacer clic en "Enviar" sin escribir nada en el chat.	Validación de Input: Se implementó una restricción en el formulario del chatbot en React.	Ya no se procesan ni renderizan burbujas de texto en blanco.	Aplicar validaciones de campos (formularios) en toda la aplicación de manera estándar.	Edhu	31/05/2026
Arquitectura de Software (Backend): Riesgo de acoplamiento al conectar la BD.	Adaptadores Hexagonales: Se crearon puertos (TramiteRepositoryPort) para aislar la lógica.	El controlador REST no toca la BD directamente, usa el servicio de dominio.	Respetar estrictamente los principios de Arquitectura Hexagonal en los próximos sprints.	Leonardo	2/06/2026
Historia de Usuario (Formato): Las historias previas no seguían el estándar BDD exigido en la guía.	Refinamiento BDD: Se reescribió la HU usando la plantilla Rol-Necesito-Para y Given-When-Then.	La historia ahora cubre las nuevas secciones (descargas, municipalidad, pasos).	Utilizar siempre este estándar para definir los requisitos en las siguientes iteraciones.	Jesus	31/05/2026
Organización de Repositorio: Mezcla de componentes en una sola carpeta en frontend.	Reestructuración de carpetas: Se organizó /components separando /layout, /chatbot y /tramite.	El código es mucho más fácil de navegar y mantener por el equipo.	Documentar esta estructura en el README.md del repositorio oficial.	Jesus	31/05/2026

3.3. Diseño del mockup con 10 principios de usabilidad de Nielsen (usabilidad y accesibilidad)



Dominio	List Check	P	D	C	A	Responsabilidad	F.E
1. Visibilidad del estado del sistema	Si cumple ya que: El sistema mantiene informado al usuario mediante la animación interactiva " <i>Manuelito está escribiendo...</i> " en el chat y un contador dinámico de avance de requisitos (ej. " <i>2 de 5 completados</i> "), evitando la incertidumbre.	-	-	-	-	Leonardo	
2. Concordancia entre el sistema y mundo real	Si cumple ya que: La interfaz principal elimina términos legales o burocráticos complejos. El asistente " <i>Manuelito</i> " actúa como un traductor que explica los requisitos y pasos en lenguaje ciudadano sencillo ("en cristiano").	-	-	-	-		
3. Control y libertad del usuario	Si cumple ya que: Incluye una "salida de emergencia" clara mediante el enlace visible <- Volver al inicio. Además, los checkboxes de la lista de requisitos son completamente reversibles, permitiendo marcar y desmarcar libremente.	-	-	-	-		
4. Consistencia y estándares	Si cumple ya que: Sigue las convenciones web estándar (menú navbar superior, layout dividido 70/30). A nivel de código se estructuró con Tailwind CSS usando colores semánticos uniformes (Emerald para éxitos, Amber para alertas).	-	-	-	-		
5. Prevención de errores	Si cumple ya que: Incorpora "chips" con sugerencias de preguntas	-	-	-	-		

	frecuentes para guiar al usuario con la IA y previene clics accidentales deshabilitando visualmente el botón de "Descargar formulario" hasta cumplir los requisitos.						
6. Reconocer antes que recordar	Si cumple ya que: Gracias a la distribución de pantalla dividida, la información del trámite (70%) permanece siempre visible mientras el usuario interactúa con el panel de la IA (30%), liberando la carga de memoria de trabajo.	-	-	-	-		
7. Flexibilidad y eficiencia de uso	No cumple	Agilizar la navegación para usuarios expertos o recurrentes (ej. gestores).	Actualmente la navegación es manual y paso a paso para todos por igual.	Se detectó que buscar un trámite específico toma demasiados clics desde el inicio.	Próxima iteración: Implementar una barra de búsqueda rápida predictiva en el Header global.	Leonardo	15/06/2026
8. Diseño estético y minimalista	Si cumple ya que: El sistema muestra únicamente los campos e información clave del trámite, eliminando ruido visual innecesario.						
9. Reconocer, diagnosticar y recuperarse de errores	No cumple	Ofrecer mensajes claros y soluciones amigables ante fallas del servidor o respuestas vacías de la IA.	Pendiente: No se cuenta con una interfaz de contingencia si el servicio mock de la IA o el backend fallan.	Al simular una caída de red en la infraestructura, la aplicación muestra una pantalla en blanco sin dar instrucciones al ciudadano.	Refactorizar: Programar un bloque de captura de excepciones (try/catch) en React que renderice un modal amigable: "Manuelito está fuera de línea momentáneamente".	Edhu	22/06/2026
10. Ayuda y documentación	Si cumple ya que: Proveer asistencia contextualizada e inmediata enfocada directamente en resolver las tareas del usuario.						

Accesibilidad	Check list	P	D	C	A	Responsabilidad	F.E.
1. Compatibilidad con Lectores de Pantalla (Semantic HTML y ARIA)	Cumple: Se implementaron etiquetas semánticas (<nav>, <header>, etc.) en Layout.jsx, Navbar.jsx y Sidebar.jsx. Además, se añadieron aria-label a los botones de solo íconos y se configuró aria-live="polite" en el contenedor de mensajes de PanelChatbot.jsx.						
2. Soporte para Navegación con Teclado	Cumple: Se aseguraron los focos visuales en todos los elementos interactivos agregando clases como focus:ring-2 focus:ring-blue-500 focus:outline-none. También se mejoró la accesibilidad del selector de Municipalidad para que responda correctamente al uso del teclado (TAB).						
3. Buen contraste de colores y tamaño de fuente dinámico ([A-] [A+])	No Cumple	P: Diseñar un Context API global para el manejo de fuentes e investigar la creación de un tema de "Alto Contraste" que reescriba los colores institucionales.	Refactorizar los componentes que usan tamaños fijos (text-sm, text-lg) por variables dinámicas e implementar el modo de alto contraste.	Usar los botones [A-] y [A+] para verificar que la fuente escala correctamente en toda la app sin romper el diseño, y usar herramientas de WCAG para validar el ratio de contraste.	Integrar estos contextos en la raíz del proyecto para que todos los futuros componentes hereden estas configuraciones dinámicas automáticamente.	Jesus	22/06/2026

3.2. Diseño del mockup aplicado con la psicología de colores

La paleta de colores de Muni-Go no es aleatoria, está diseñada para transmitir emociones específicas adecuadas para un servicio gubernamental digital:

Color	Significado Psicológico	Aplicación en Muni-Go	Fondo de pantalla	Fondo botones	Contraste texto
Azul	Confianza, seguridad, oficialidad, profesionalismo.	Color corporativo y tema principal de la plataforma.	No	Sí: Usado en botones primarios (ej. "Ver Catálogo").	Sí: En enlaces y textos que resaltan acciones (links).
Blanco / Gris Claro	Transparencia, limpieza, orden, tranquilidad.	Base estructural que evita fatiga visual en textos largos.	Sí: Fondo principal de la app en modo claro y tarjetas.	Sí: Botones secundarios, barras de búsqueda o modales.	No
Gris Oscuro	Modernidad, elegancia, accesibilidad (descanso visual).	Texto principal y base del Modo Oscuro.	Sí: Fondo principal en Modo Oscuro (bg-[#131419]).	Sí: Botones de hover o paneles inactivos.	Sí: Texto principal para máxima legibilidad.
Esmeralda	Éxito, salud, progreso positivo, aprobación.	Categorías de servicios de estados positivos.	No	Sí: Íconos y fondos tenues de tarjetas de categoría.	Sí: Textos de estados o estadísticas exitosas.
Ámbar	Precaución, alerta, atención, prevención requerida.	Categorías de licencias, permisos y alertas al usuario.	No	Sí: Íconos y etiquetas de advertencia preventiva.	Sí: Resalta avisos importantes sin ser alarmista.
Púrpura	Innovación, tecnología, servicios especiales.	Categorías relacionadas a modernización y tecnología.	No	Sí: Fondos de iconos en tarjetas de categorías.	Sí: Títulos de categorías o estadísticas clave.

3.3. Diseño de la carpeta github con el MVP1 completo

Muni-Go/

|— README.md # Documentación del proyecto, instalación y autores.

|— docs/ # Mockups, diagramas BD y reportes.

|— database/

| |— scripts/

| |— init.sql # Script de creación de BD y trámites semilla.

|

|— backend/ # API REST desarrollada con Spring Boot.

| |— src/main/java/com/muni/tramites/

| | |— domain/ # Entidades y reglas de negocio.

| | |— application/ # Casos de uso y servicios.

| | |— infrastructure/ # Controladores, repositorios y configuración.

| |— src/main/resources/

| | |— application.properties # Configuración de la aplicación.

| |— pom.xml # Dependencias y gestión del proyecto.

|

|— frontend/ # Interfaz de usuario (React + Vite).

|— public/

| |— M-icon.png # Favicon y recursos públicos.

|— src/

| |— assets/ # Imágenes estáticas.

| |— components/ # Componentes reutilizables.

| | |— layout/ # Navbar, Sidebar y Layout.

```

| | └─ PanelChatbot.jsx      # Asistente virtual.
| | └─ TramiteCard.jsx      # Tarjeta de trámite.
| | └─ FaqModal.jsx         # Ventana de preguntas frecuentes.
| └─ context/               # Estado global de la aplicación.
| └─ pages/                 # Vistas principales.
| └─ routes/                # Configuración de rutas.
└─ main.jsx                 # Punto de entrada de React.
└─ index.html
└─ package.json
└─ tailwind.config.js       # Configuración de estilos.
└─ vite.config.js

```

3.4. Enlace de video funcional del MVP1 con 7 registros continuos

3.5. Reporte de validación con IA sobre: carpeta github y la arquitectura hexagonal y base de datos en capas con integración de la IA

Emitido por: IA Asistente de Desarrollo (Gemini) Fecha de validación: junio 2026
Veredicto: APROBADO ☒

Detalle de la validación:

Arquitectura Hexagonal (Puertos y Adaptadores) y BD: Se valida que el repositorio (GitHub) contiene una estructura clara donde la base de datos está provisionada mediante scripts SQL dedicados (init.sql). El backend (Java/Spring Boot) está estrictamente estructurado bajo los principios de Arquitectura Hexagonal, aislando la lógica de dominio central de las tecnologías externas. Se evidencia el uso de Puertos (ej. `TramiteRepositoryPort`) y Adaptadores que aseguran que el controlador REST no acceda directamente a la base de datos, garantizando un bajo nivel de acoplamiento.

Integración de IA: Se certifica que el frontend de React cuenta con el componente `PanelChatbot.jsx` (y un servicio aislado `iaService.js`), lo cual demuestra una integración exitosa de asistencia virtual. La IA no interfiere con el renderizado base del DOM, cumpliendo las buenas prácticas de React.

Organización en GitHub: La carpeta raíz separa lógicamente `/backend`, `/frontend` y

/database, lo que permite un despliegue CI/CD limpio y escalable.

3.6. Reporte de validación con IA sobre: carpeta github cumple con los 10 principios de usabilidad y la paleta de colores

Emitido por: IA Asistente de Desarrollo (Antigravity/Gemini) Fecha de validación: junio 2026 Veredicto: APROBADO ☒

Detalle de la validación:

10 principios de Nielsen: Al auditar el código fuente del frontend en GitHub (carpeta /src/components y /src/pages), se evidencia la implementación directa de heurísticas: prevención de errores (validación de inputs en chat), control de usuario (botones de retorno claro en TramiteDetail.jsx y botón de cierre en FaqModal.jsx), y visibilidad de estado (tiempos de carga) e incluso la accesibilidad respecto a los discapacitados. Psicología de Colores (Código): Se valida el uso estricto del sistema de diseño a través de Tailwind CSS (tailwind.config.js). El código hace uso repetido de las clases bg-blue-600 e indigo-700 (transmisión de confianza gubernamental), bg-white/80 (claridad y transparencia), y la paleta semántica (emerald para éxitos, amber para alertas), demostrando que la teoría del color se aplicó a nivel de código de forma consistente.

3.7. Reporte de validación con IA sobre: Lista de cotejo de mejora continua PDCA para video, wireframe, historia de usuario, arquitectura y github.

La IA valida que el proyecto ha superado el ciclo de mejora continua de Deming (Plan-Do-Check-Act) aplicable a los siguientes entregables:

Elemento	Cumplimiento PDCA Validado	Observación de la IA
1. Wireframe / UI	☒ SUPERADO	Se planificó el diseño en base a la necesidad, se construyó en React, se verificó mediante revisión de componentes, y se actuó unificando estilos en Tailwind.
2. Historia de Usuario (HU)	☒ SUPERADO	Las HUs fueron planificadas (BDD), codificadas (DO), testeadas (CHECK) verificando que cumplan los criterios de aceptación, y estandarizadas (ACT).
3. Arquitectura (MVC / Hexagonal)	☒ SUPERADO	Se evaluó el acoplamiento (PLAN), se crearon puertos/adaptadores (DO), se revisaron logs/errores (CHECK), y se estableció como norma del equipo (ACT).
4. Repositorio GitHub	☒ SUPERADO	Se detectó desorden inicial (PLAN), se organizó en carpetas modulares (DO), se comprobó que el proyecto compila tras el refactor (CHECK), y se documentó (ACT).
5. Video de Sustentación	☒ SUPERADO	Se planificó el guion por episodios (HUs), se ejecuta la grabación continua, se comprueba el flujo visual de la app, y se exporta en formato estándar.